

Министерство образования и науки РФ
федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет (институт) Информационных технологий и компьютерных систем
Кафедра Информатики и вычислительной техники

КУРСОВОЙ ПРОЕКТ

по дисциплине Программирование
на тему Разработка модели работы звукового синтезатора и компьютерной игры

Пояснительная записка

Шифр проекта 020-КП-09.03.01-ИВТ-123-24-ПЗ

Студента (ки) Шмидта Антона Владиславовича
фамилия, имя, отчество полностью

Курс 1 Группа ИВТ-244

Направление (специальность) 09.03.01 –
Информатика и вычислительная техника
код, наименование

Руководитель к.т.н., доцент
ученая степень, звание

Шафеева О.П.
фамилия, инициалы

Выполнил (а) 28.05.2025 ШШ
дата, подпись студента (ки)

К защите 28.05.2025
дата, подпись руководителя

Выполнение и подготовка к защите КП (КР)	Защита КП (КР)	Итоговый рейтинг
<u>59</u>	<u>38</u>	<u>97</u>

Проект (работа) защищен (а) с оценкой отлично

Омск 2025

Шафеева О.П.

Содержание

Текстовые формулировки задач	3
Математическая формулировка задачи	4
Общая схема алгоритмов.....	5
Детальные схемы алгоритмов	6
Код	20
Разработка пользовательского интерфейса	38
Тестирование	42
Список литературы	45

Текстовые формулировки задач

Разработать схему алгоритма, написать и отладить следующие программы:

1. Анимационная заставка.
2. Вывод информации об авторе работы.
3. Звуковой синтезатор. Составить программу для воспроизведения мелодии, сыгранной на клавиатуре компьютера. Каждой клавише ставится в соответствие некоторая нота, длительность звучания ноты определяется длительностью нажатия клавиши.
4. Игровая программа «Морской бой». На поле 10x10 позиций стоят невидимые корабли: 4 корабля по одной клетке, 3 корабля по две клетки, 2 корабля по три клетки и один корабль в четыре клетки. Необходимо поразить каждую из клеток кораблей. Позиции указываются русскими буквами от А до К и цифрами от 1 до 10. Конфигурация и положение кораблей на поле выбираются с помощью датчика случайных чисел. Если клетка корабля угадана играющим верно, она отмечается крестиком; в противном случае точкой.

Математическая формулировка задачи

Задание 1: Создание звукового синтезатора — это задача, заключающаяся в программной генерации звуковых волн заданной частоты в режиме реального времени в ответ на действия пользователя, например, нажатие клавиш на клавиатуре. Каждая клавиша соответствует определённой ноте и, следовательно, конкретной частоте. При нажатии клавиши начинается синтез звука с помощью математической модели синусоидальной волны, параметры которой зависят от частоты ноты и текущей фазы сигнала. Звук формируется как поток числовых значений, соответствующих уровням амплитуды аудиосигнала, и передаётся на звуковое устройство компьютера с фиксированной частотой дискретизации. Во время удержания клавиши волна продолжается, и при её отпуске генерация прекращается. Таким образом, синтезатор превращает цифровые команды (нажатия клавиш) в непрерывный звуковой поток, имитируя поведение настоящего музыкального инструмента.

Задание 2: Игра «Морской бой» представляет собой дискретную стратегическую задачу на двумерной решётке размером 10×10 . На этом поле размещается конечное множество кораблей различной длины. Каждый корабль занимает строго горизонтальную или вертикальную последовательность клеток, не выходящую за границы поля. Корабли не должны соприкасаться друг с другом ни сторонами, ни углами, то есть для любого корабля все клетки вокруг него (включая диагональные) должны быть свободны.

Математически каждое размещение корабля можно представить как подмножество клеток решётки, а всё поле кораблей — как объединение этих подмножеств. Задача генерации поля заключается в построении такого семейства непересекающихся и не прикасающихся подмножеств заданных размеров, полностью укладывающихся в границы сетки.

Игровой процесс сводится к пошаговому выбору клеток, по которым совершается "выстрел". После каждого выстрела игрок получает информацию о том, попал ли он в корабль, промахнулся или уничтожил весь корабль. Основная цель игры — определить расположение всех кораблей на поле, полностью поразив каждую их клетку. Таким образом, игра представляет собой задачу поиска скрытых подмножеств (кораблей) на дискретной плоскости при помощи последовательных логических проверок с ограниченной обратной связью.

Общая схема алгоритмов

Курсовой проект состоит из следующих частей:

- Анимированная заставка;
- Об авторе;
- Звуковой синтезатор;
- Игра «Морской бой»;
- Выход.

Для выполнения этих задач было разработано меню, управление в котором осуществляется путем нажатия стрелок на клавиатуре. Для вывода нужной подпрограммы следует нажать на клавишу Enter.

На рисунке 1 представлена общая схема алгоритма работы меню. На клавиатуре с помощью стрелок выбирается пункт и нажимается Enter. Для возврата из подпрограммы обратно в меню нужно нажать клавишу Esc. Если в меню пользователь выбирает пункт ВЫХОД, то программа завершает свою работу. Программа предоставляет информацию об авторе, выводит программу для воспроизведения звуков, генерирует игру «Морской бой», а также выводит динамическую заставку.

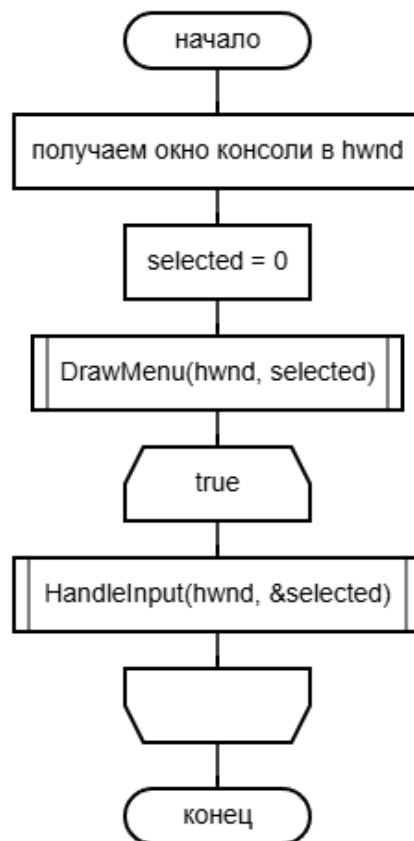


Рисунок 1 – схема алгоритма главной функции.

Детальные схемы алгоритмов

В программе были объявлены глобальные переменные в файле «functions.cpp»:

```
const int MENU_ITEMS = 5;
const char* menuItems[MENU_ITEMS] = {
    "1. Заставка",
    "2. Об авторе",
    "3. Синтезатор",
    "4. Морской бой",
    "5. Выход"
};
```

Так же был создан заголовочный файл «functions.h», который подключался к каждому файлу проекта с помощью «#include "functions.h"»:

```
#pragma once
#define _USE_MATH_DEFINES
#include <windows.h>
#include <string>
#include <cstring>
#include <cmath>
#include <iomanip>
#include <sstream>
#include <SDL3/SDL.h>
#include <stdlib.h>
#include <time.h>

using namespace std;

extern const int MENU_ITEMS;
extern const char* menuItems[];

void DrawMenu(HWND hwnd, int selected);
void HandleInput(HWND hwnd, int* selected);
void ShowIntro(HWND hwnd);
void ShowAbout(HWND hwnd);
void ShowSynth(HWND hwnd);
void ShowGame(HWND hwnd);
```

На рисунке 2 приведена схема отрисовки главного меню программы.

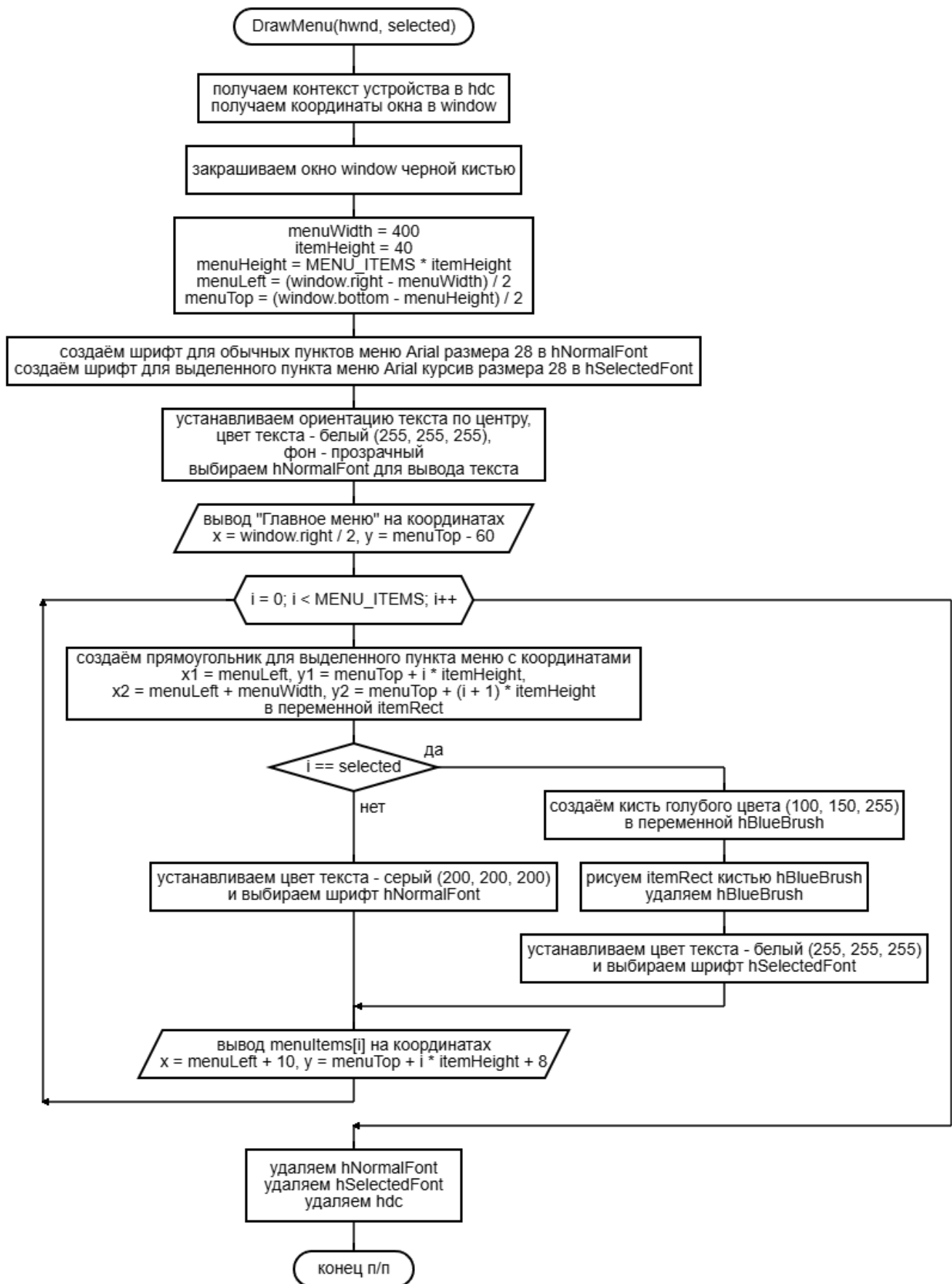


Рисунок 2 – схема алгоритма вывода главного меню.

На рисунке 3 показан алгоритм обработки ввода клавиш в главном меню программы.

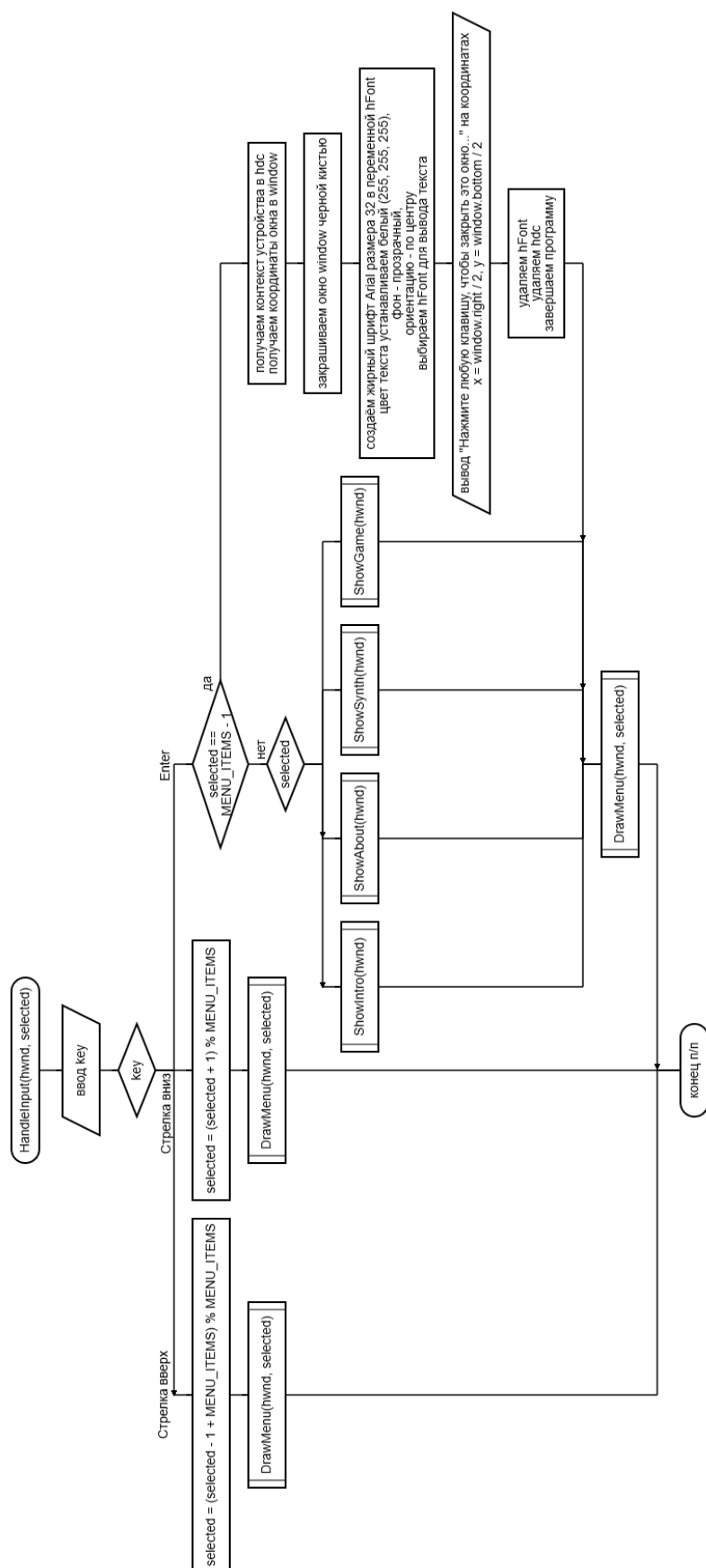


Рисунок 3 – схема алгоритма работы главного меню.

На рисунке 4 показана схема отрисовки динамической заставки программы.

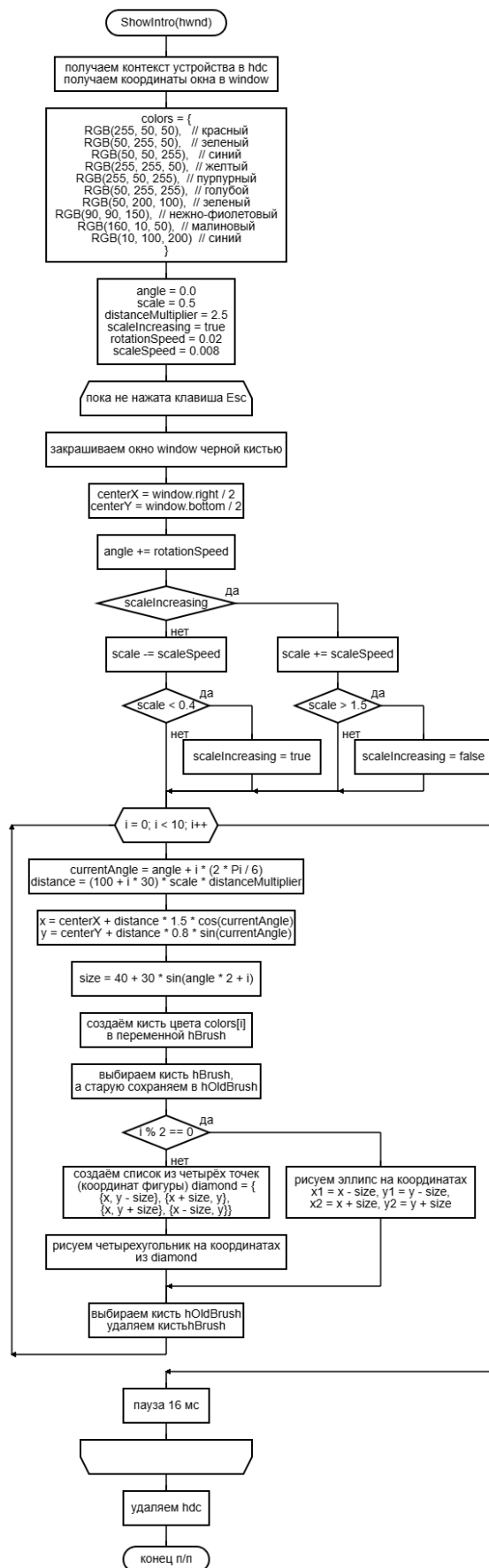


Рисунок 4 – схема алгоритма динамической заставки.

На рисунке 5 приведен алгоритм для вывода информации об авторе программы.

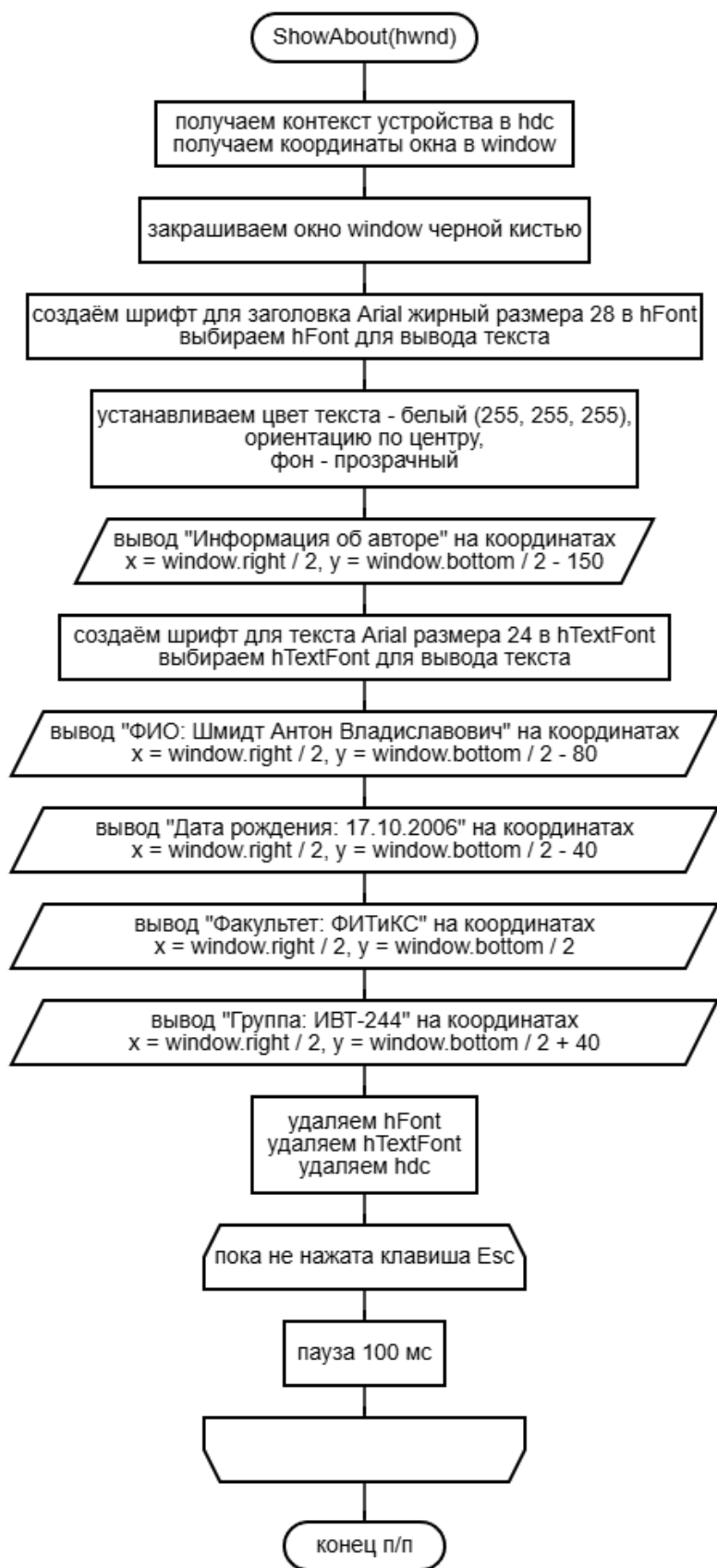


Рисунок 5 – схема алгоритма вывода информации об авторе.

На рисунке 6 показан алгоритм обработки нажатий и воспроизведения соответствующих звуков в подпрограмме звукового синтезатора.

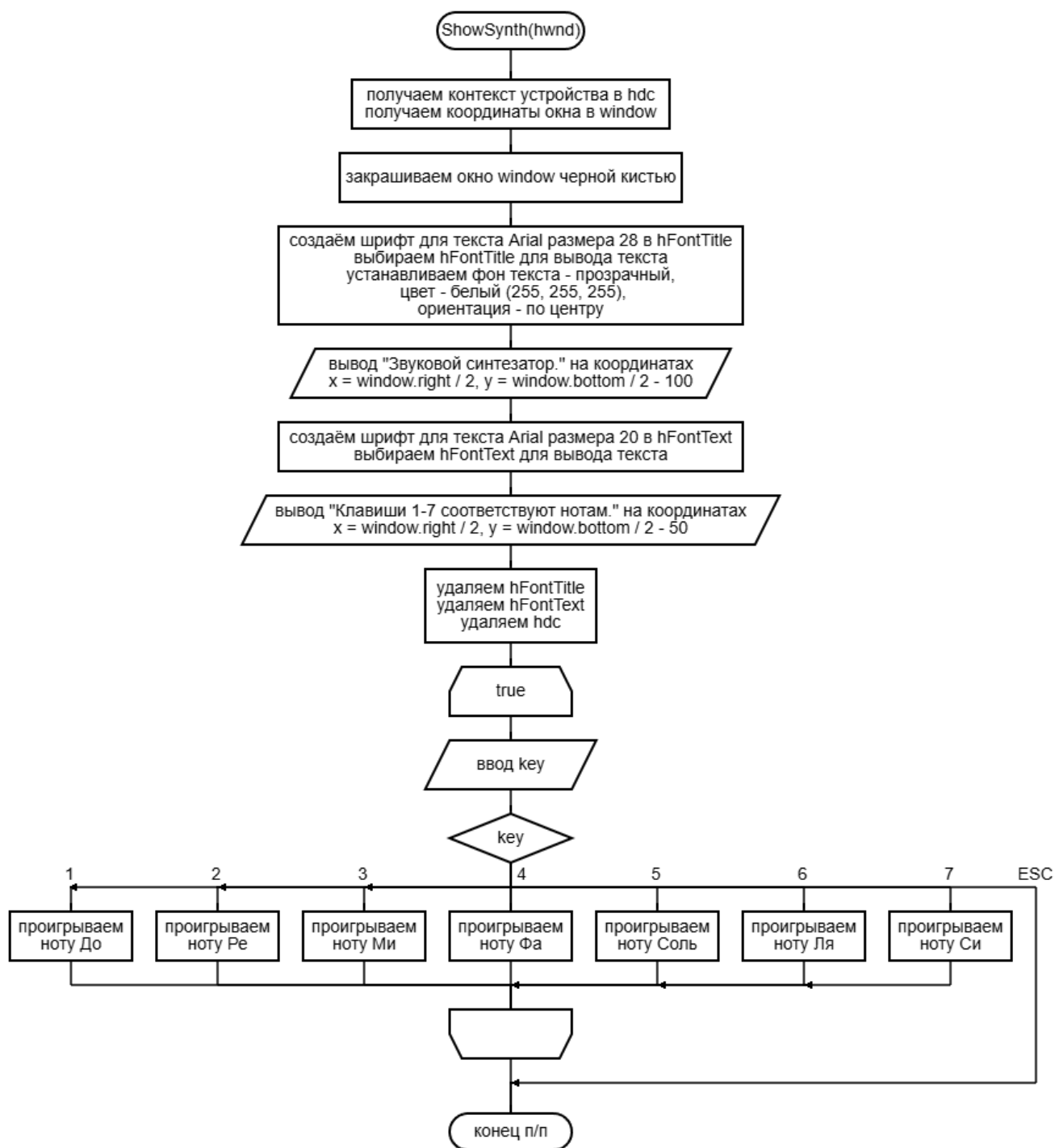


Рисунок 6 – схема алгоритма звукового синтезатора.

Для игры «Морской бой» на уровне программы были объявлены глобальные переменные: $GRID_SIZE = 10$, $ships[GRID_SIZE][GRID_SIZE]$, $shots[GRID_SIZE][GRID_SIZE]$, $selectedCol = -1$, $selectedRow = -1$; которые оперировались в функциях ниже.

На рисунке 7 представлен алгоритм для генерации случайно расположенных кораблей на поле размером 10 на 10 клеток.

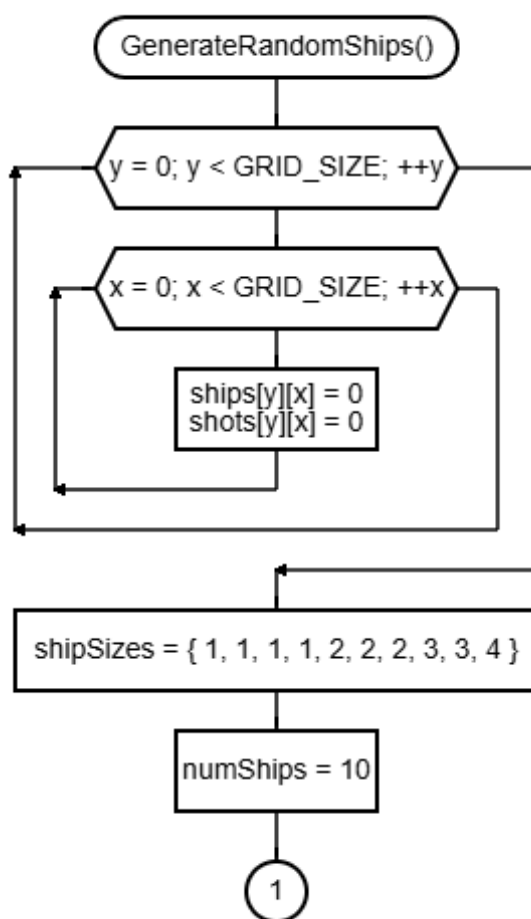
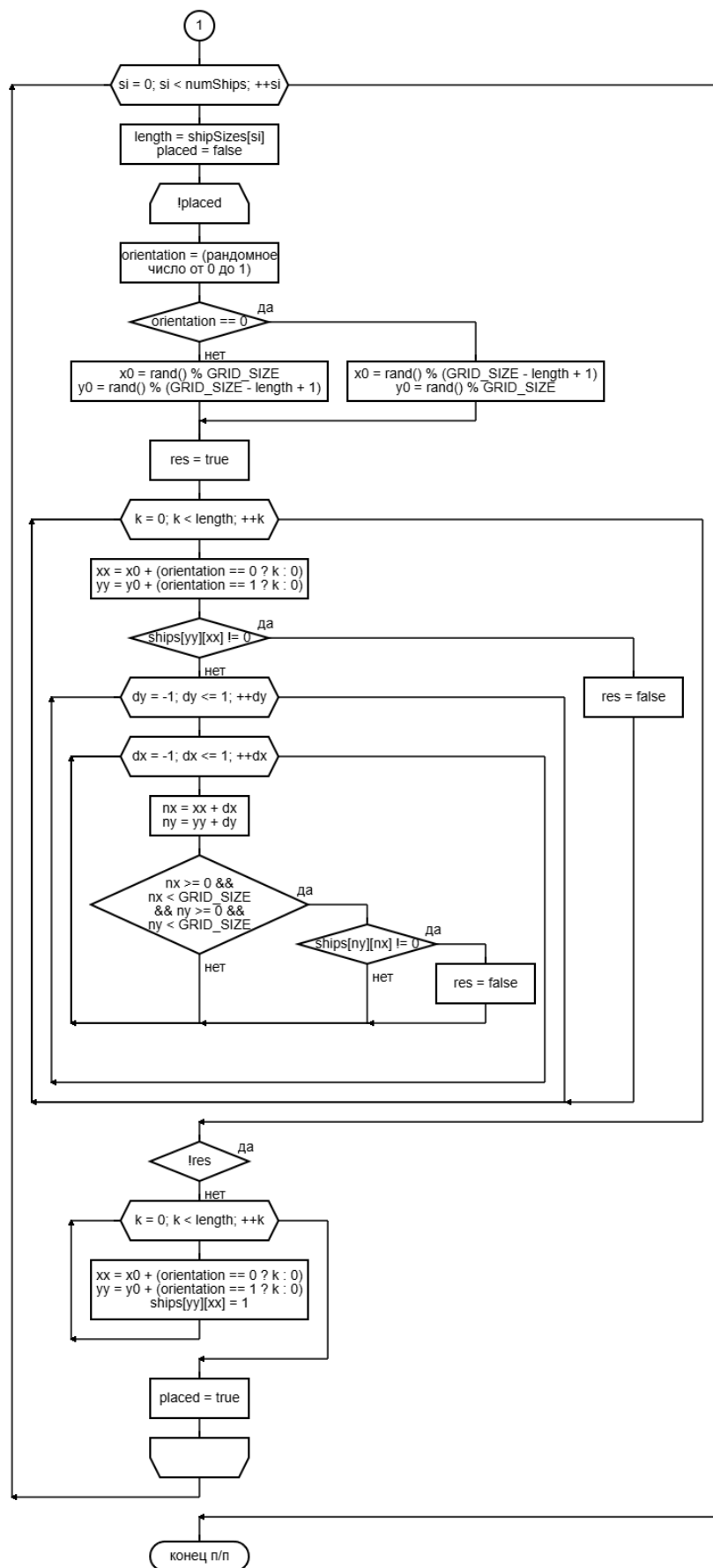


Рисунок 7 – схема алгоритма генерации положения кораблей для игры «Морской бой».



Продолжение рисунка 7 – схема алгоритма генерации положения кораблей для игры «Морской бой».

На рисунке 8 показан алгоритм отрисовки поля и интерфейса игры «Морской бой».

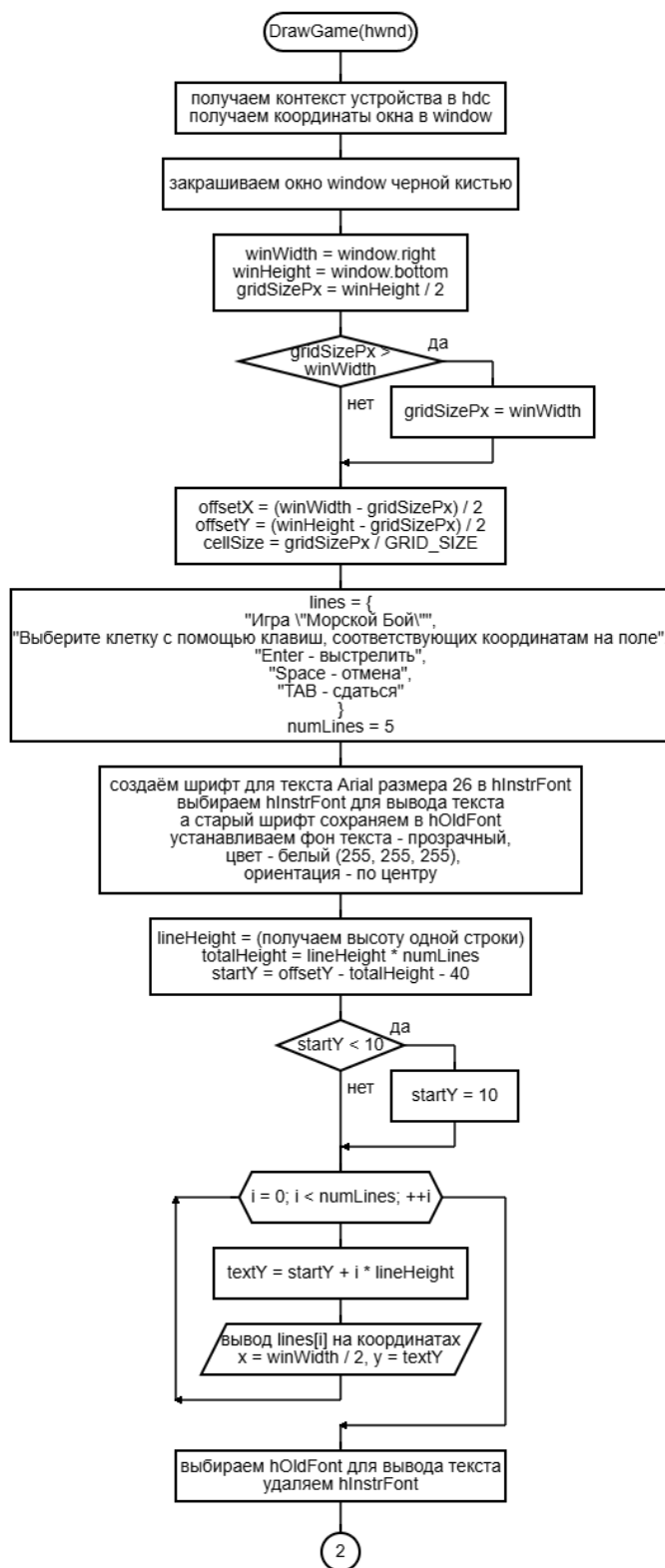
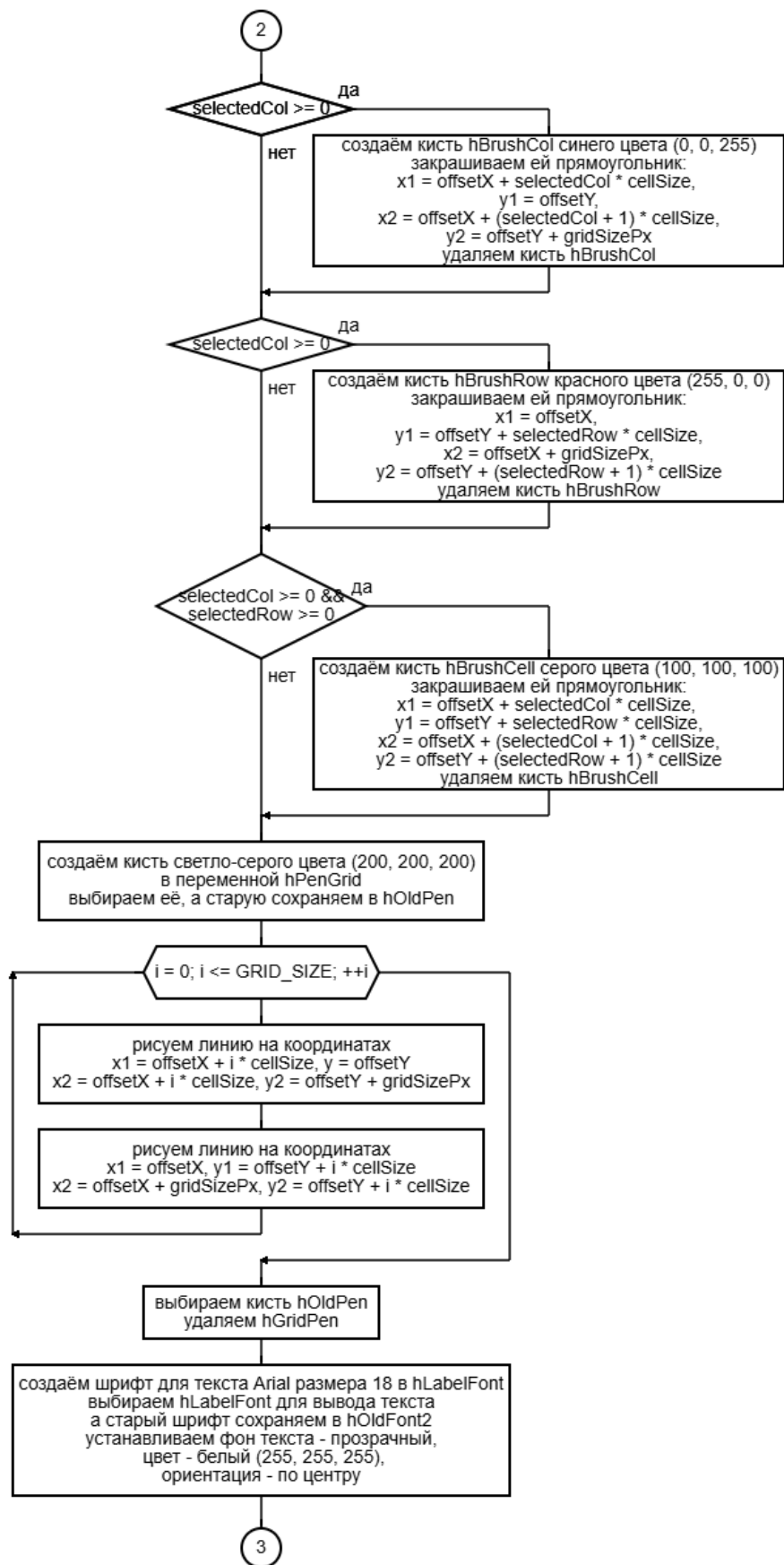
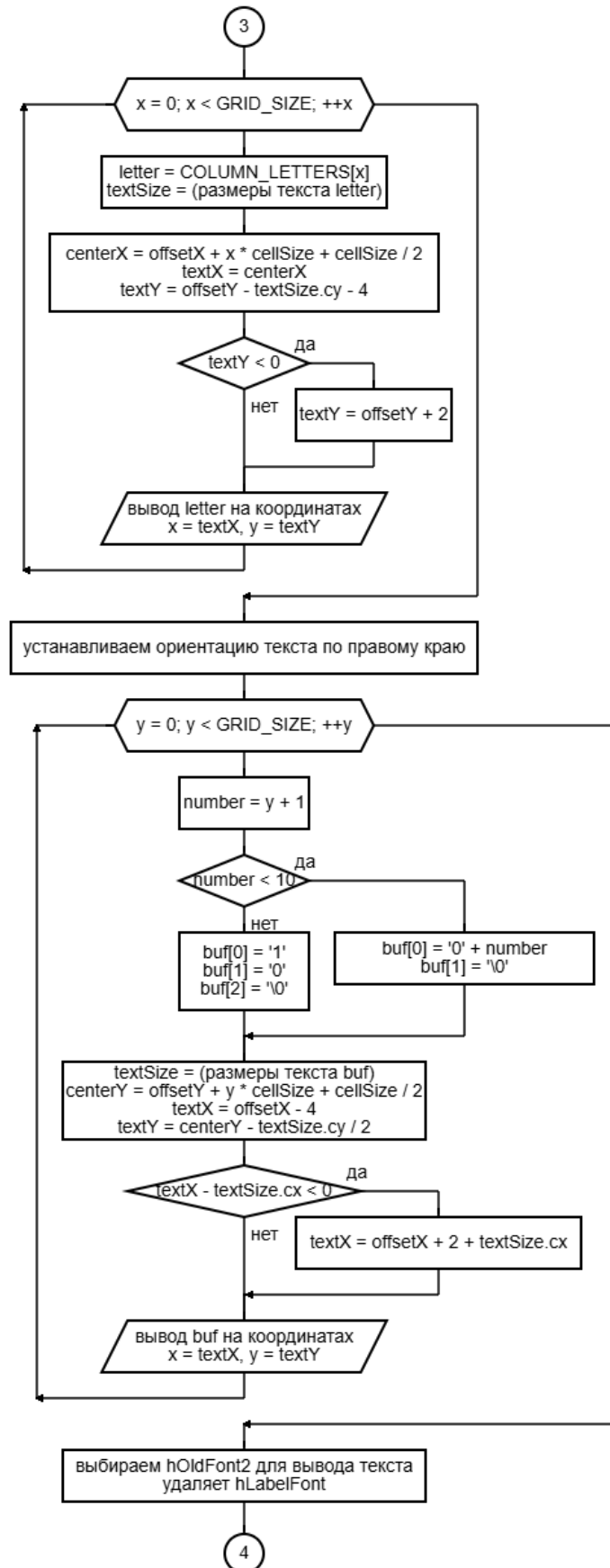


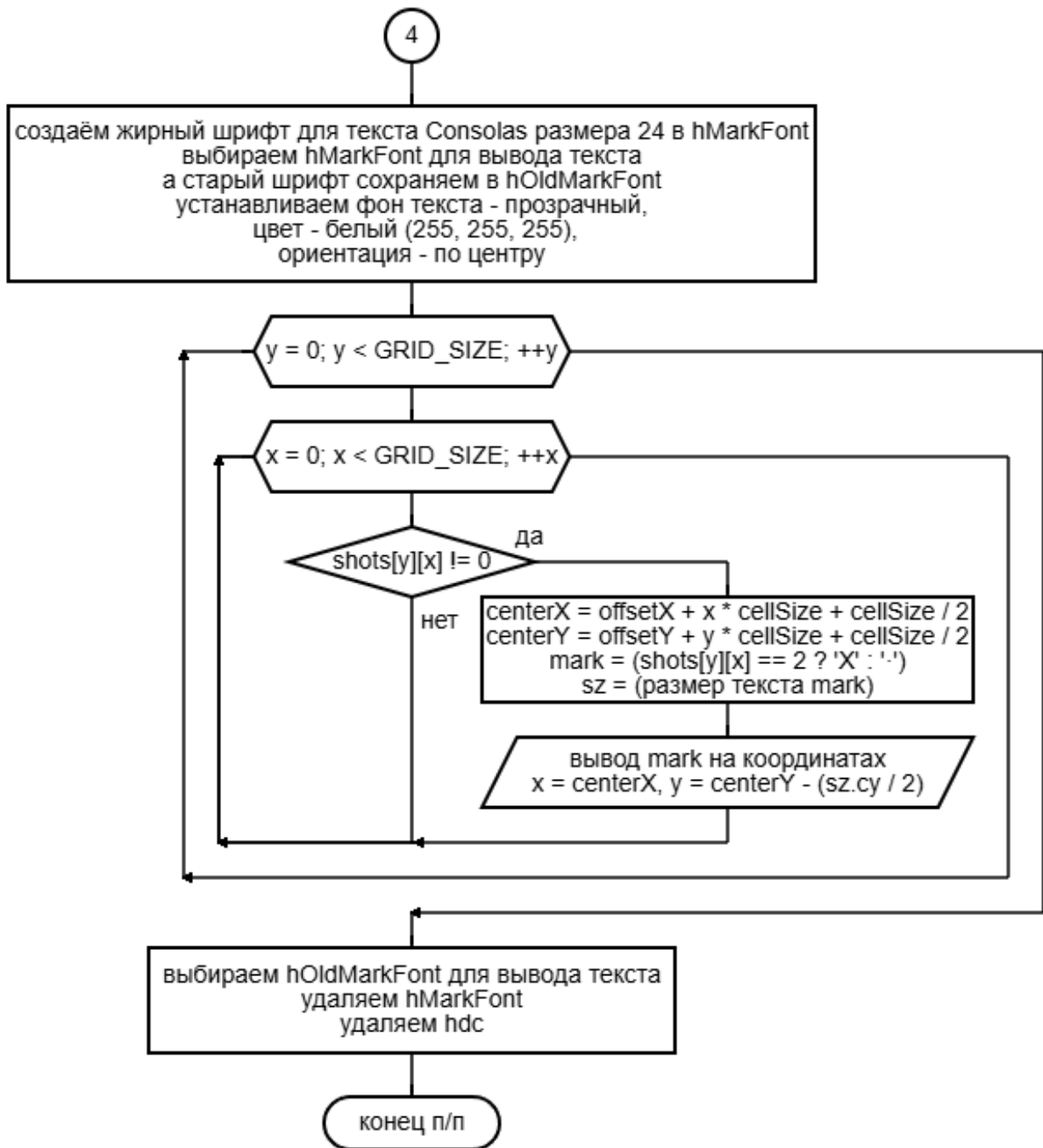
Рисунок 8 – схема алгоритма генерации интерфейса игры «Морской бой».



Продолжение рисунка 8 – схема алгоритма генерации интерфейса игры
«Морской бой».



Продолжение рисунка 8 – схема алгоритма генерации интерфейса игры «Морской бой».



Продолжение рисунка 8 – схема алгоритма генерации интерфейса игры
 «Морской бой».

На рисунке 9 представлена схема раскрытия положения всех кораблей на поле игры в случае, если игрок решит сдаться.

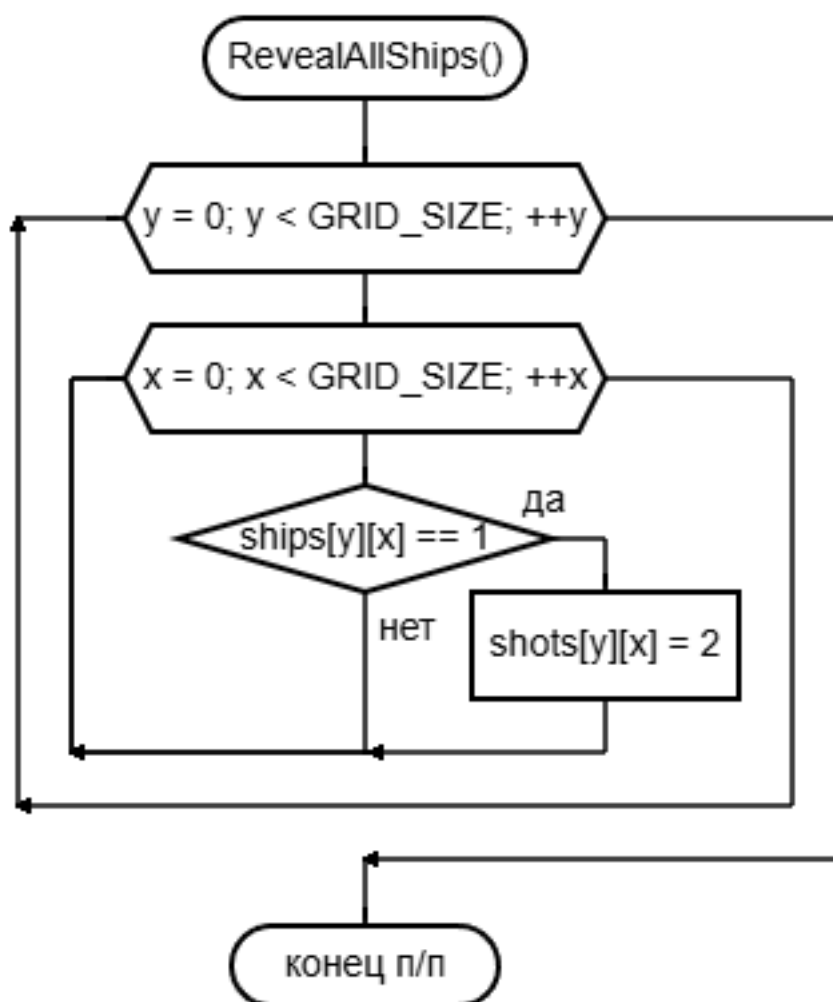


Рисунок 9 – схема алгоритма отметки всех кораблей для игры «Морской бой».

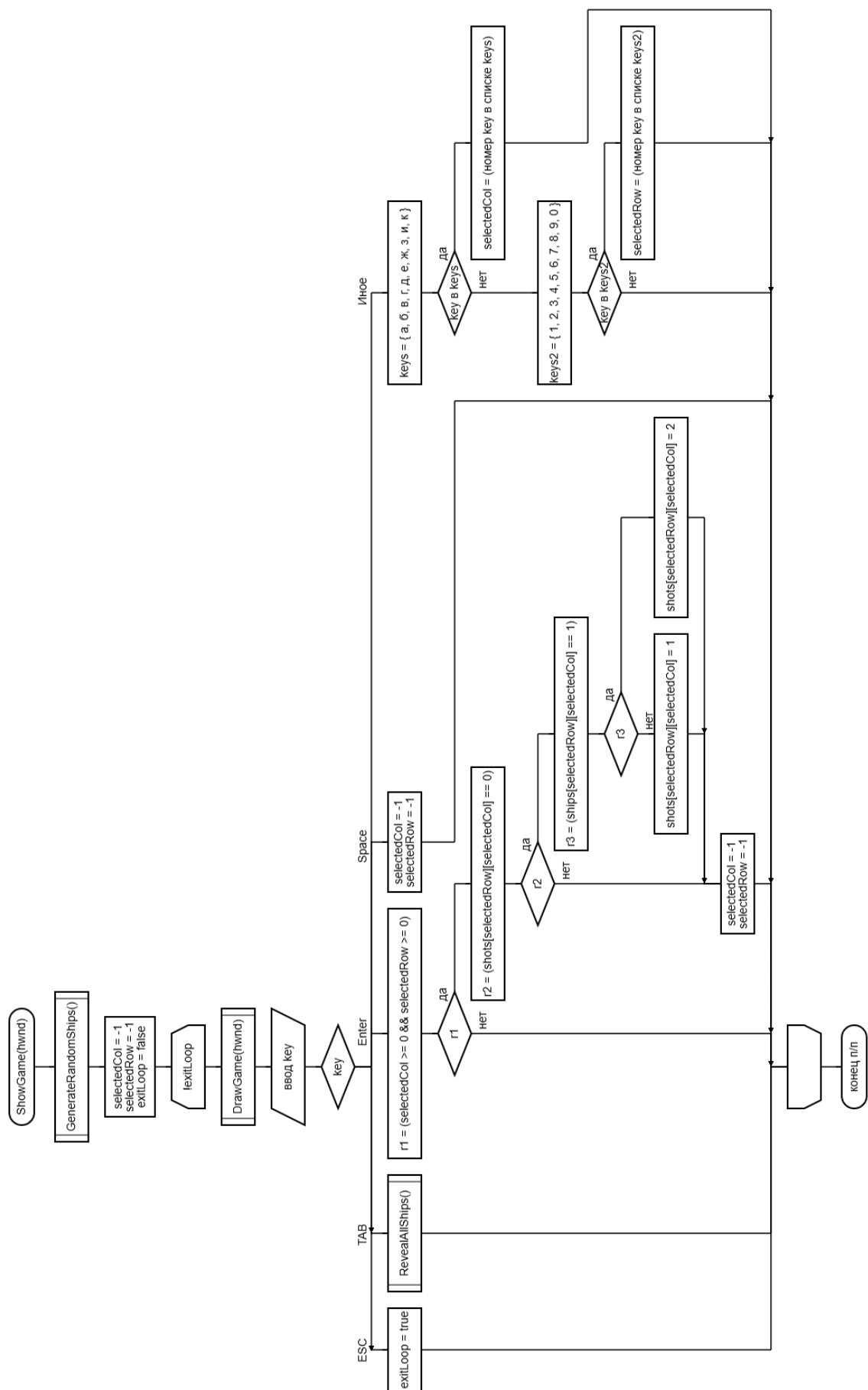


Рисунок 10 – схема алгоритма вывода интерфейса игры «Морской бой».

Код

functions.cpp

```
#include "functions.h"

const int MENU_ITEMS = 5;
const char* menuItems[MENU_ITEMS] = {
    "1. Заставка",
    "2. Об авторе",
    "3. Синтезатор",
    "4. Морской бой",
    "5. Выход"
};
```

functions.h

```
#pragma once
#define _USE_MATH_DEFINES
#include <windows.h>
#include <string>
#include <cstring>
#include <cmath>
#include <iomanip>
#include <sstream>
#include <SDL3/SDL.h>
#include <stdlib.h>
#include <time.h>

using namespace std;

extern const int MENU_ITEMS;
extern const char* menuItems[];

void DrawMenu(HWND hwnd, int selected);
void HandleInput(HWND hwnd, int* selected);
void ShowIntro(HWND hwnd);
void ShowAbout(HWND hwnd);
void ShowSynth(HWND hwnd);
void ShowGame(HWND hwnd);
```

main.cpp

```
#include "functions.h"

int main() {
    HWND hwnd = GetConsoleWindow();
    ShowWindow(hwnd, SW_MAXIMIZE);

    CONSOLE_CURSOR_INFO cci = { 1, FALSE };
    SetConsoleCursorInfo(GetStdHandle(STD_OUTPUT_HANDLE), &cci);

    int selected = 0;
    DrawMenu(hwnd, selected);
}
```

```

    while (true) {
        HandleInput(hwnd, &selected);
    }

    return 0;
}

```

menu.cpp

```
#include "functions.h"
```

```

void DrawMenu(HWND hwnd, int selected) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);

    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    const int menuWidth = 400;
    const int itemHeight = 40;
    const int menuHeight = MENU_ITEMS * itemHeight;

    const int menuLeft = (window.right - menuWidth) / 2;
    const int menuTop = (window.bottom - menuHeight) / 2;

    HFONT hNormalFont = CreateFont(28, 0, 0, 0, FW_NORMAL, FALSE,
    FALSE, FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Arial");
    HFONT hSelectedFont = CreateFont(28, 0, 0, 0, FW_NORMAL, TRUE,
    FALSE, FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Arial");

    SetTextAlign(hdc, TA_CENTER);
    SetTextColor(hdc, RGB(255, 255, 255));
    SetBkMode(hdc, TRANSPARENT);
    SelectObject(hdc, hNormalFont);
    TextOutA(hdc, window.right / 2, menuTop - 60, "Главное меню",
12);

    SetTextAlign(hdc, TA_LEFT);
    for (int i = 0; i < MENU_ITEMS; i++) {
        RECT itemRect = {
            menuLeft,
            menuTop + i * itemHeight,
            menuLeft + menuWidth,
            menuTop + (i + 1) * itemHeight
        };
    };
}

```

```

        if (i == selected) {
            HBRUSH hBlueBrush = CreateSolidBrush(
255));
            FillRect(hdc, &itemRect, hBlueBrush);
            DeleteObject(hBlueBrush);

            SetTextColor(hdc, RGB(255, 255, 255));
            SelectObject(hdc, hSelectedFont);
        }
        else {
            SetTextColor(hdc, RGB(200, 200, 200));
            SelectObject(hdc, hNormalFont);
        }

        TextOutA(hdc, menuLeft + 10, menuTop + i * itemHeight + 8,
menuItems[i], (int)strlen(menuItems[i]));
    }

    DeleteObject(hNormalFont);
    DeleteObject(hSelectedFont);
    ReleaseDC(hwnd, hdc);
}

void HandleInput(HWND hwnd, int* selected) {
    if (GetAsyncKeyState(VK_UP) & 0x8000) {
        *selected = (*selected - 1 + MENU_ITEMS) % MENU_ITEMS;
        DrawMenu(hwnd, *selected);
        Sleep(150);
    }
    else if (GetAsyncKeyState(VK_DOWN) & 0x8000) {
        *selected = (*selected + 1) % MENU_ITEMS;
        DrawMenu(hwnd, *selected);
        Sleep(150);
    }
    else if (GetAsyncKeyState(VK_RETURN) & 0x8000) {
        if (*selected == MENU_ITEMS - 1) {
            HDC hdc = GetDC(hwnd);
            RECT window;
            GetClientRect(hwnd, &window);

            FillRect(hdc, &window,
(HBRUSH)GetStockObject(BLACK_BRUSH));

            SetTextColor(hdc, RGB(255, 255, 255));
            SetBkMode(hdc, TRANSPARENT);
            HFONT hFont = CreateFont(32, 0, 0, 0, FW_BOLD, FALSE,
FALSE, FALSE,
                DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
                CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
                DEFAULT_PITCH, L"Arial");
            SelectObject(hdc, hFont);

```

```

        SetTextAlign(hdc, TA_CENTER);

        const char* msg = "Нажмите любую клавишу, чтобы закрыть
ЭТО ОКНО...";
        TextOutA(hdc, window.right / 2, window.bottom / 2, msg,
(int)strlen(msg));

        DeleteObject(hFont);
        ReleaseDC(hwnd, hdc);
        exit(0);
    }

    switch (*selected) {
        case 0: ShowIntro(hwnd); break;
        case 1: ShowAbout(hwnd); break;
        case 2: ShowSynth(hwnd); break;
        case 3: ShowGame(hwnd); break;
    }

    DrawMenu(hwnd, *selected);
}
}

```

intro.cpp

```
#include "functions.h"
```

```

void ShowIntro(HWND hwnd) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);

    const COLORREF colors[] = {
        RGB(255, 50, 50),    // красный
        RGB(50, 255, 50),    // зеленый
        RGB(50, 50, 255),    // синий
        RGB(255, 255, 50),    // желтый
        RGB(255, 50, 255),    // пурпурный
        RGB(50, 255, 255),    // голубой
        RGB(50, 200, 100),    // зеленый
        RGB(90, 90, 150),     // нежно-фиолетовый
        RGB(160, 10, 50),     // малиновый
        RGB(10, 100, 200)     // синий
    };

    float angle = 0.0f;
    float scale = 0.5f;
    float distanceMultiplier = 2.5f;
    bool scaleIncreasing = true;
    const float rotationSpeed = 0.02f;
    const float scaleSpeed = 0.008f;
}

```

```

while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) {
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    const int centerX = window.right / 2;
    const int centerY = window.bottom / 2;

    angle += rotationSpeed;
    if (scaleIncreasing) {
        scale += scaleSpeed;
        if (scale > 1.5f) scaleIncreasing = false;
    }
    else {
        scale -= scaleSpeed;
        if (scale < 0.4f) scaleIncreasing = true;
    }

    for (int i = 0; i < 10; i++) {
        float currentAngle = angle + i * (2 * M_PI / 6);
        int distance = (100 + i * 30) * scale * distanceMultiplier;

        int x = centerX + distance * 1.5f * cos(currentAngle);
        int y = centerY + distance * 0.8f * sin(currentAngle);

        int size = 40 + 30 * sin(angle * 2 + i);

        HBRUSH hBrush = CreateSolidBrush(colors[i]);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        if (i % 2 == 0) {
            Ellipse(hdc, x - size, y - size, x + size, y + size);
        }
        else {
            POINT diamond[4] = {
                {x, y - size},
                {x + size, y},
                {x, y + size},
                {x - size, y}
            };
            Polygon(hdc, diamond, 4);
        }

        SelectObject(hdc, hOldBrush);
        DeleteObject(hBrush);
    }

    Sleep(16);

    GdiFlush();
}

ReleaseDC(hwnd, hdc);
}

```


about.cpp

```
#include "functions.h"
```

```
void ShowAbout(HWND hwnd) {
    HDC hdc = GetDC(hwnd);

    RECT window;
    GetClientRect(hwnd, &window);
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    HFONT hFont = CreateFont(28, 0, 0, 0, FW_BOLD, FALSE, FALSE, FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hFont);

    SetTextColor(hdc, RGB(255, 255, 255));
    SetTextAlign(hdc, TA_CENTER);
    SetBkMode(hdc, TRANSPARENT);

    TextOutA(hdc, window.right / 2, window.bottom / 2 - 150,
        "Информация об авторе", 20);

    HFONT hTextFont = CreateFont(24, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
        FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hTextFont);

    TextOutA(hdc, window.right / 2, window.bottom / 2 - 80, "ФИО:
Шмидт Антон Владиславович", 30);
    TextOutA(hdc, window.right / 2, window.bottom / 2 - 40, "Дата
рождения: 17.10.2006", 25);
    TextOutA(hdc, window.right / 2, window.bottom / 2, "Факультет:
ФИТиКС", 17);
    TextOutA(hdc, window.right / 2, window.bottom / 2 + 40, "Группа:
ИБТ-244", 15);

    DeleteObject(hFont);
    DeleteObject(hTextFont);
    ReleaseDC(hwnd, hdc);

    while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) Sleep(100);
}
```

synth.cpp

```
#include "functions.h"
```

```
static constexpr int SAMPLE_RATE = 44100;
```

```

static constexpr int NUM_KEYS = 7;
static constexpr float GLOBAL_VOLUME = 0.3f;
static constexpr int CHUNK_SIZE = 512;

static const double noteFreq[NUM_KEYS] = {
    261.63, 293.66, 329.63, 349.23, 392.00, 440.00, 493.88
};

static bool keyActive[NUM_KEYS] = { false };
static double phase[NUM_KEYS] = { 0.0 };

static void SDLCALL FeedAudio(void* userdata, SDL_AudioStream*
astream, int additional_amount, int total_amount) {
    int samplesNeeded = additional_amount /
static_cast<int>(sizeof(float));

    while (samplesNeeded > 0) {
        int chunk = (samplesNeeded > CHUNK_SIZE) ? CHUNK_SIZE :
samplesNeeded;
        float buffer[CHUNK_SIZE];

        for (int i = 0; i < chunk; ++i) {
            float sampleValue = 0.0f;
            for (int k = 0; k < NUM_KEYS; ++k) {
                if (keyActive[k]) {
                    double angular = 2.0 * M_PI * noteFreq[k] /
SAMPLE_RATE;
                    sampleValue += static_cast<float>(sin(phase[k])
* GLOBAL_VOLUME);
                    phase[k] += angular;
                    if (phase[k] > 2.0 * M_PI) {
                        phase[k] -= 2.0 * M_PI;
                    }
                }
            }
            if (sampleValue > 1.0f) sampleValue = 1.0f;
            else if (sampleValue < -1.0f) sampleValue = -1.0f;
            buffer[i] = sampleValue;
        }

        SDL_PutAudioStreamData(astream, buffer, chunk *
sizeof(float));
        samplesNeeded -= chunk;
    }
}

void ShowSynth(HWND hwnd)
{
    HDC hdc = GetDC(hwnd);

    RECT window;
    GetClientRect(hwnd, &window);

```

```

FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

HFONT hFontTitle = CreateFontW(
    28, 0, 0, 0, FW_BOLD, FALSE, FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial"
);
SelectObject(hdc, hFontTitle);

SetTextColor(hdc, RGB(255, 255, 255));
SetTextAlign(hdc, TA_CENTER);
SetBkMode(hdc, TRANSPARENT);

TextOutA(hdc, window.right / 2, window.bottom / 2 - 100,
"Звуковой синтезатор.", 19);

HFONT hFontText = CreateFontW(
    20, 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial"
);
SelectObject(hdc, hFontText);

TextOutA(hdc, window.right / 2, window.bottom / 2 - 50, "Клавиши
1-7 соответствуют нотам.", 29);

DeleteObject(hFontTitle);
DeleteObject(hFontText);
ReleaseDC(hwnd, hdc);

SDL_Init(SDL_INIT_AUDIO | SDL_INIT_VIDEO);

SDL_Window* sdlWindow = SDL_CreateWindow(
    "Piano",
    640, 480,
    0
);

SDL_SetWindowOpacity(sdlWindow, 0.0f);
SDL_RaiseWindow(sdlWindow);

SDL_AudioSpec spec;
SDL_zero(spec);
spec.freq = SAMPLE_RATE;
spec.format = SDL_AUDIO_F32;
spec.channels = 1;

SDL_AudioStream* stream = SDL_OpenAudioDeviceStream(
    SDL_AUDIO_DEVICE_DEFAULT_PLAYBACK,
    &spec,

```

```

        FeedAudio,
        nullptr
    );

    SDL_ResumeAudioStreamDevice(stream);

    bool running = true;
    SDL_Event event;
    while (running) {
        while (SDL_PollEvent(&event)) {
            switch (event.type) {
                case SDL_EVENT_QUIT:
                    running = false;
                    break;
                case SDL_EVENT_KEY_DOWN: {
                    SDL_Keycode key = event.key.scancode;
                    switch (key) {
                        case SDL_SCANCODE_ESCAPE:
                            running = false;
                            break;
                        case SDL_SCANCODE_1: keyActive[0] = true;
                        case SDL_SCANCODE_2: keyActive[1] = true;
                        case SDL_SCANCODE_3: keyActive[2] = true;
                        case SDL_SCANCODE_4: keyActive[3] = true;
                        case SDL_SCANCODE_5: keyActive[4] = true;
                        case SDL_SCANCODE_6: keyActive[5] = true;
                        case SDL_SCANCODE_7: keyActive[6] = true;
                        default: break;
                    }
                    break;
                }
                case SDL_EVENT_KEY_UP: {
                    SDL_Keycode key = event.key.scancode;
                    switch (key) {
                        case SDL_SCANCODE_1:
                            keyActive[0] = false;
                            phase[0] = 0.0;
                            break;
                        case SDL_SCANCODE_2:
                            keyActive[1] = false;
                            phase[1] = 0.0;
                            break;
                        case SDL_SCANCODE_3:
                            keyActive[2] = false;
                            phase[2] = 0.0;
                    }
                }
            }
        }
    }

```

```

        break;
    case SDL_SCANCODE_4:
        keyActive[3] = false;
        phase[3] = 0.0;
        break;
    case SDL_SCANCODE_5:
        keyActive[4] = false;
        phase[4] = 0.0;
        break;
    case SDL_SCANCODE_6:
        keyActive[5] = false;
        phase[5] = 0.0;
        break;
    case SDL_SCANCODE_7:
        keyActive[6] = false;
        phase[6] = 0.0;
        break;
    default: break;
}
break;
}
default: break;
}
}
SDL_Delay(10);
}

SDL_DestroyAudioStream(stream);
SDL_DestroyWindow(sdlWindow);
SDL_Quit();
}

```

game.cpp

```
#include "functions.h"
```

```
const int GRID_SIZE = 10;
```

```
const COLORREF COLOR_HIGHLIGHT_COL = RGB(0, 0, 255);
const COLORREF COLOR_HIGHLIGHT_ROW = RGB(255, 0, 0);
const COLORREF COLOR_HIGHLIGHT_CELL = RGB(100, 100, 100);
const COLORREF COLOR_GRID_LINE = RGB(200, 200, 200);
```

```
static int ships[GRID_SIZE][GRID_SIZE];
static int shots[GRID_SIZE][GRID_SIZE];
```

```
static int selectedCol = -1;
static int selectedRow = -1;
```

```
static const wchar_t COLUMN_LETTERS[GRID_SIZE] = {
    L'A', L'B', L'B', L'Г', L'Д', L'E', L'Ж', L'З', L'И', L'K'
};
```

```

void GenerateRandomShips() {
    for (int y = 0; y < GRID_SIZE; ++y) {
        for (int x = 0; x < GRID_SIZE; ++x) {
            ships[y][x] = 0;
            shots[y][x] = 0;
        }
    }

    const int shipSizes[] = {
        1, 1, 1, 1,
        2, 2, 2,
        3, 3,
        4
    };
    const int numShips = sizeof(shipSizes) / sizeof(shipSizes[0]);

    srand((unsigned)time(NULL));

    auto canPlaceAt = [&](int x0, int y0, int length, int
orientation) {
        for (int k = 0; k < length; ++k) {
            int xx = x0 + (orientation == 0 ? k : 0);
            int yy = y0 + (orientation == 1 ? k : 0);

            if (ships[yy][xx] != 0)
                return false;

            for (int dy = -1; dy <= 1; ++dy) {
                for (int dx = -1; dx <= 1; ++dx) {
                    int nx = xx + dx;
                    int ny = yy + dy;
                    if (nx >= 0 && nx < GRID_SIZE && ny >= 0 && ny <
GRID_SIZE) {
                        if (ships[ny][nx] != 0)
                            return false;
                    }
                }
            }
        }
        return true;
    };

    for (int si = 0; si < numShips; ++si) {
        int length = shipSizes[si];
        bool placed = false;

        while (!placed) {
            int orientation = rand() % 2;
            int x0, y0;

            if (orientation == 0) {

```

```

        x0 = rand() % (GRID_SIZE - length + 1);
        y0 = rand() % GRID_SIZE;
    }
    else {
        x0 = rand() % GRID_SIZE;
        y0 = rand() % (GRID_SIZE - length + 1);
    }

    if (!canPlaceAt(x0, y0, length, orientation))
        continue;

    for (int k = 0; k < length; ++k) {
        int xx = x0 + (orientation == 0 ? k : 0);
        int yy = y0 + (orientation == 1 ? k : 0);
        ships[yy][xx] = 1;
    }
    placed = true;
}
}
}

void DrawGame(HWND hwnd) {
    HDC hdc = GetDC(hwnd);

    RECT window;
    GetClientRect(hwnd, &window);
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    int winWidth = window.right;
    int winHeight = window.bottom;

    int gridSizePx = winHeight / 2;
    if (gridSizePx > winWidth) {
        gridSizePx = winWidth;
    }

    int offsetX = (winWidth - gridSizePx) / 2;
    int offsetY = (winHeight - gridSizePx) / 2;

    int cellSize = gridSizePx / GRID_SIZE;

    {
        const wchar_t* lines[] = {
            L"Игра \"Морской Бой\"",
            L"Выберите клетку с помощью клавиш, соответствующих",
            L"координатам на поле",
            L"Enter - выстрелить",
            L"Space - отмена",
            L"TAB - сдаться"
        };
        const int numLines = sizeof(lines) / sizeof(lines[0]);
    }
}

```

```

HFONT hInstrFont = CreateFont(
    26, 0, 0, 0, FW_NORMAL,
    FALSE, FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH | FF_DONTCARE,
    L"Arial"
);
HFONT hOldFont = (HFONT)SelectObject(hdc, hInstrFont);
SetTextColor(hdc, RGB(255, 255, 255));
SetBkMode(hdc, TRANSPARENT);
SetTextAlign(hdc, TA_CENTER | TA_TOP);

TEXTMETRIC tm;
GetTextMetrics(hdc, &tm);
int lineHeight = tm.tmHeight;

int totalHeight = lineHeight * numLines;

int startY = offsetY - totalHeight - 40;
if (startY < 10) {
    startY = 10;
}

for (int i = 0; i < numLines; ++i) {
    int textY = startY + i * lineHeight;
    TextOutW(hdc, winWidth / 2, textY, lines[i],
(int)wcslen(lines[i]));
}

SelectObject(hdc, hOldFont);
DeleteObject(hInstrFont);
}

if (selectedCol >= 0) {
    HBRUSH hBrushCol = CreateSolidBrush(COLOR_HIGHLIGHT_COL);
    RECT rcCol = {
        offsetX + selectedCol * cellSize,
        offsetY,
        offsetX + (selectedCol + 1) * cellSize,
        offsetY + gridSizePx
    };
    FillRect(hdc, &rcCol, hBrushCol);
    DeleteObject(hBrushCol);
}

if (selectedRow >= 0) {
    HBRUSH hBrushRow = CreateSolidBrush(COLOR_HIGHLIGHT_ROW);
    RECT rcRow = {
        offsetX,
        offsetY + selectedRow * cellSize,
        offsetX + gridSizePx,

```



```

        offsetY + (selectedRow + 1) * cellSize
    };
    FillRect(hdc, &rcRow, hBrushRow);
    DeleteObject(hBrushRow);
}

if (selectedCol >= 0 && selectedRow >= 0) {
    HBRUSH hBrushCell = CreateSolidBrush(COLOR_HIGHLIGHT_CELL);
    RECT rcCell = {
        offsetX + selectedCol * cellSize,
        offsetY + selectedRow * cellSize,
        offsetX + (selectedCol + 1) * cellSize,
        offsetY + (selectedRow + 1) * cellSize
    };
    FillRect(hdc, &rcCell, hBrushCell);
    DeleteObject(hBrushCell);
}

HPEN hPenGrid = CreatePen(PS_SOLID, 1, COLOR_GRID_LINE);
HPEN hOldPen = (HPEN)SelectObject(hdc, hPenGrid);
for (int i = 0; i <= GRID_SIZE; ++i) {
    MoveToEx(hdc, offsetX + i * cellSize, offsetY, NULL);
    LineTo(hdc, offsetX + i * cellSize, offsetY + gridSizePx);

    MoveToEx(hdc, offsetX, offsetY + i * cellSize, NULL);
    LineTo(hdc, offsetX + gridSizePx, offsetY + i * cellSize);
}
SelectObject(hdc, hOldPen);
DeleteObject(hPenGrid);

HFONT hLabelFont = CreateFont(
    18, 0, 0, 0, FW_BOLD,
    FALSE, FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH | FF_DONTCARE,
    L"Arial"
);
HFONT hOldFont2 = (HFONT)SelectObject(hdc, hLabelFont);
SetTextColor(hdc, RGB(255, 255, 255));
SetBkMode(hdc, TRANSPARENT);
SetTextAlign(hdc, TA_CENTER | TA_TOP);

for (int x = 0; x < GRID_SIZE; ++x) {
    wchar_t letter = COLUMN_LETTERS[x];
    SIZE textSize;
    GetTextExtentPoint32W(hdc, &letter, 1, &textSize);

    int centerX = offsetX + x * cellSize + cellSize / 2;
    int textX = centerX;
    int textY = offsetY - textSize.cy - 4;
    if (textY < 0) {

```

```

        textY = offsetY + 2;
    }
    TextOutW(hdc, textX, textY, &letter, 1);
}

SetTextAlign(hdc, TA_RIGHT | TA_TOP);
for (int y = 0; y < GRID_SIZE; ++y) {
    wchar_t buf[3] = { 0 };
    int number = y + 1;
    if (number < 10) {
        buf[0] = L'0' + number;
        buf[1] = L'\\0';
    }
    else {
        buf[0] = L'1';
        buf[1] = L'0';
        buf[2] = L'\\0';
    }

    SIZE textSize;
    GetTextExtentPoint32W(hdc, buf, wcslen(buf), &textSize);

    int centerY = offsetY + y * cellSize + cellSize / 2;
    int textX = offsetX - 4;
    int textY = centerY - textSize.cy / 2;
    if (textX - textSize.cx < 0) {
        textX = offsetX + 2 + textSize.cx;
    }
    TextOutW(hdc, textX, textY, buf, (int)wcslen(buf));
}

SelectObject(hdc, hOldFont2);
DeleteObject(hLabelFont);

HFONT hMarkFont = CreateFont(
    24, 0, 0, 0, FW_BOLD,
    FALSE, FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH | FF_DONTCARE,
    L"Consolas"
);
HFONT hOldMarkFont = (HFONT)SelectObject(hdc, hMarkFont);
SetTextColor(hdc, RGB(255, 255, 255));
SetBkMode(hdc, TRANSPARENT);
SetTextAlign(hdc, TA_CENTER | TA_TOP);

for (int y = 0; y < GRID_SIZE; ++y) {
    for (int x = 0; x < GRID_SIZE; ++x) {
        if (shots[y][x] != 0) {
            int centerX = offsetX + x * cellSize + cellSize / 2;
            int centerY = offsetY + y * cellSize + cellSize / 2;

```

```

        wchar_t mark = (shots[y][x] == 2 ? L'X' : L'·');
        SIZE sz;
        GetTextExtentPoint32W(hdc, &mark, 1, &sz);
        TextOutW(
            hdc,
            centerX,
            centerY - (sz.cy / 2),
            &mark,
            1
        );
    }
}

SelectObject(hdc, hOldMarkFont);
DeleteObject(hMarkFont);

ReleaseDC(hwnd, hdc);
}

void RevealAllShips() {
    for (int y = 0; y < GRID_SIZE; ++y) {
        for (int x = 0; x < GRID_SIZE; ++x) {
            if (ships[y][x] == 1) {
                shots[y][x] = 2;
            }
        }
    }
}

void ShowGame(HWND hwnd) {
    GenerateRandomShips();
    selectedCol = -1;
    selectedRow = -1;

    bool exitLoop = false;
    const int KEY_POLL_DELAY_MS = 50;

    SHORT prevKeyStateCols[GRID_SIZE] = { 0 };
    SHORT prevKeyStateRows[10] = { 0 };
    SHORT prevKeyStateSpace = 0;
    SHORT prevKeyStateEnter = 0;
    SHORT prevKeyStateEsc = 0;
    SHORT prevKeyStateTab = 0;

    const int COL_KEYS[GRID_SIZE] = {
        0x46, // f a
        0xBC, // , 6
        0x44, // d b
        0x55, // u r
        0x4C, // l л
    };

```

```

    0x54, // t e
    0xBA, // ; ж
    0x50, // p з
    0x42, // b и
    0x52, // r к
};

while (!exitLoop) {
    DrawGame(hwnd);

    SHORT escState = GetAsyncKeyState(VK_ESCAPE);
    if ((escState & 0x8000) && !(prevKeyStateEsc & 0x8000)) {
        exitLoop = true;
    }
    prevKeyStateEsc = escState;

    SHORT tabState = GetAsyncKeyState(VK_TAB);
    if ((tabState & 0x8000) && !(prevKeyStateTab & 0x8000)) {
        RevealAllShips();
    }
    prevKeyStateTab = tabState;

    for (int i = 0; i < GRID_SIZE; ++i) {
        SHORT keyState = GetAsyncKeyState(COL_KEYS[i]);
        if ((keyState & 0x8000) && !(prevKeyStateCols[i] &
0x8000)) {
            selectedCol = i;
        }
        prevKeyStateCols[i] = keyState;
    }

    const int ROW_KEYS[10] = {
        0x31, // 1
        0x32, // 2
        0x33, // 3
        0x34, // 4
        0x35, // 5
        0x36, // 6
        0x37, // 7
        0x38, // 8
        0x39, // 9
        0x30 // 0
    };

    for (int i = 0; i < 10; ++i) {
        SHORT keyState = GetAsyncKeyState(ROW_KEYS[i]);
        if ((keyState & 0x8000) && !(prevKeyStateRows[i] &
0x8000)) {
            if (i < 9) {
                selectedRow = i;
            }
            else {

```

```

        selectedRow = 9;
    }
}
prevKeyStateRows[i] = keyState;
}

SHORT spaceState = GetAsyncKeyState(VK_SPACE);
if ((spaceState & 0x8000) && !(prevKeyStateSpace & 0x8000))
{
    selectedCol = -1;
    selectedRow = -1;
}
prevKeyStateSpace = spaceState;

SHORT enterState = GetAsyncKeyState(VK_RETURN);
if ((enterState & 0x8000) && !(prevKeyStateEnter & 0x8000))
{
    if (selectedCol >= 0 && selectedRow >= 0) {
        if (shots[selectedRow][selectedCol] == 0) {
            if (ships[selectedRow][selectedCol] == 1) {
                shots[selectedRow][selectedCol] = 2;
            }
            else {
                shots[selectedRow][selectedCol] = 1;
            }
        }
        selectedCol = -1;
        selectedRow = -1;
    }
}
prevKeyStateEnter = enterState;

Sleep(50);
}
}

```

Разработка пользовательского интерфейса

На следующих рисунках представлен интерфейс всех состояний программы.

Рисунок 11 показывает главное меню программы, из которого производится запуск всех подпрограмм.

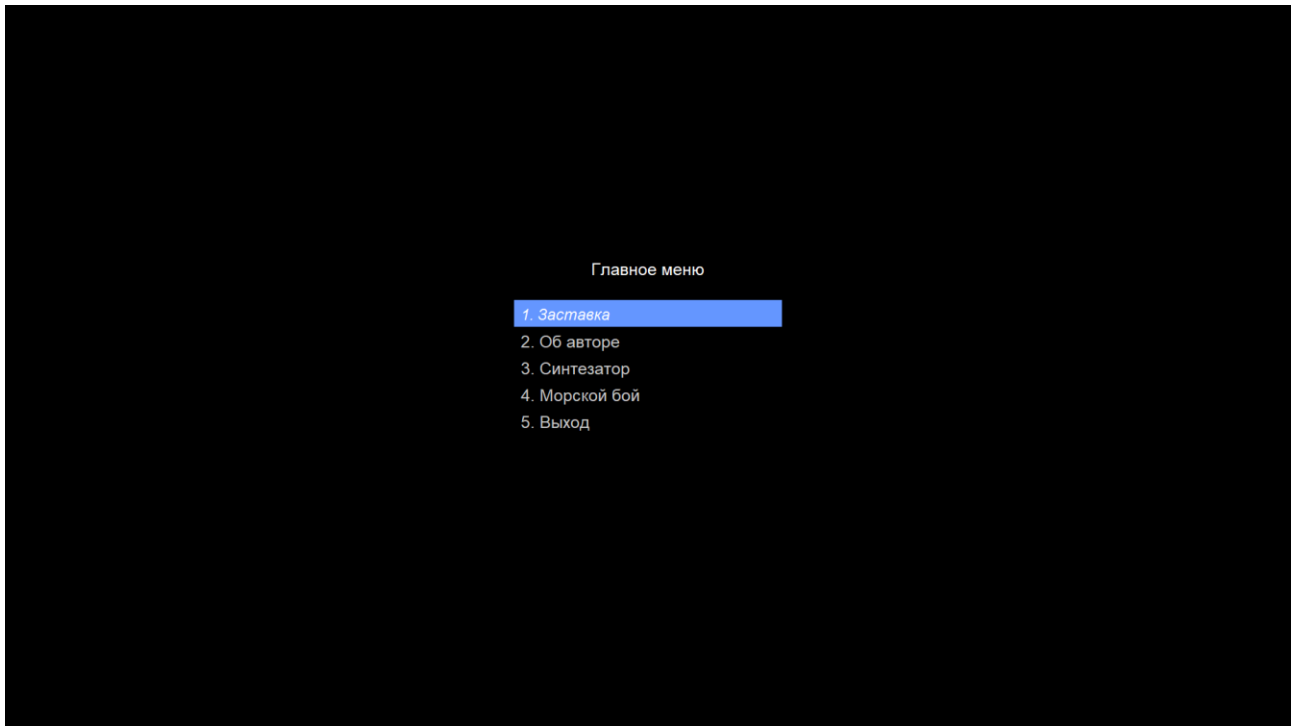


Рисунок 11 – интерфейс главного меню.

На рисунке 12 показан кадр из анимированной заставки программы.

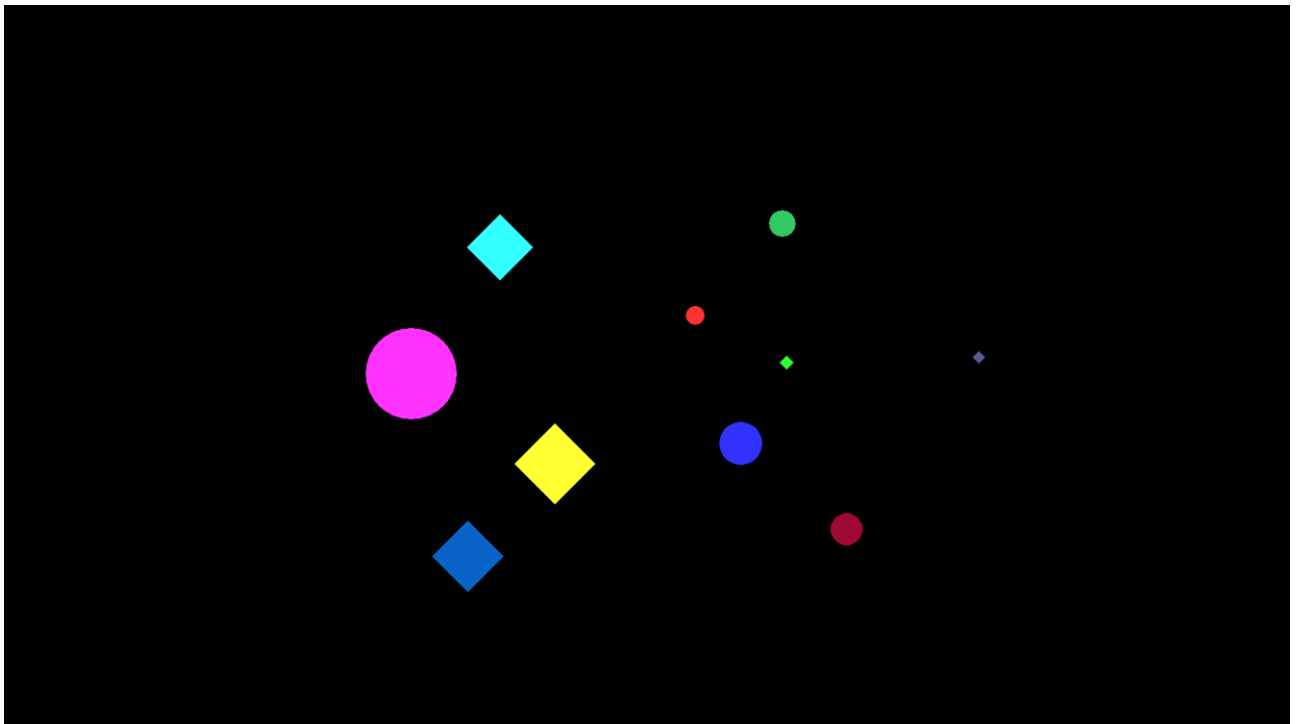


Рисунок 12 – динамическая заставка.

На рисунке 13 показан вывод информации об авторе программы, а именно: ФИО, даты рождения, факультета и группы.

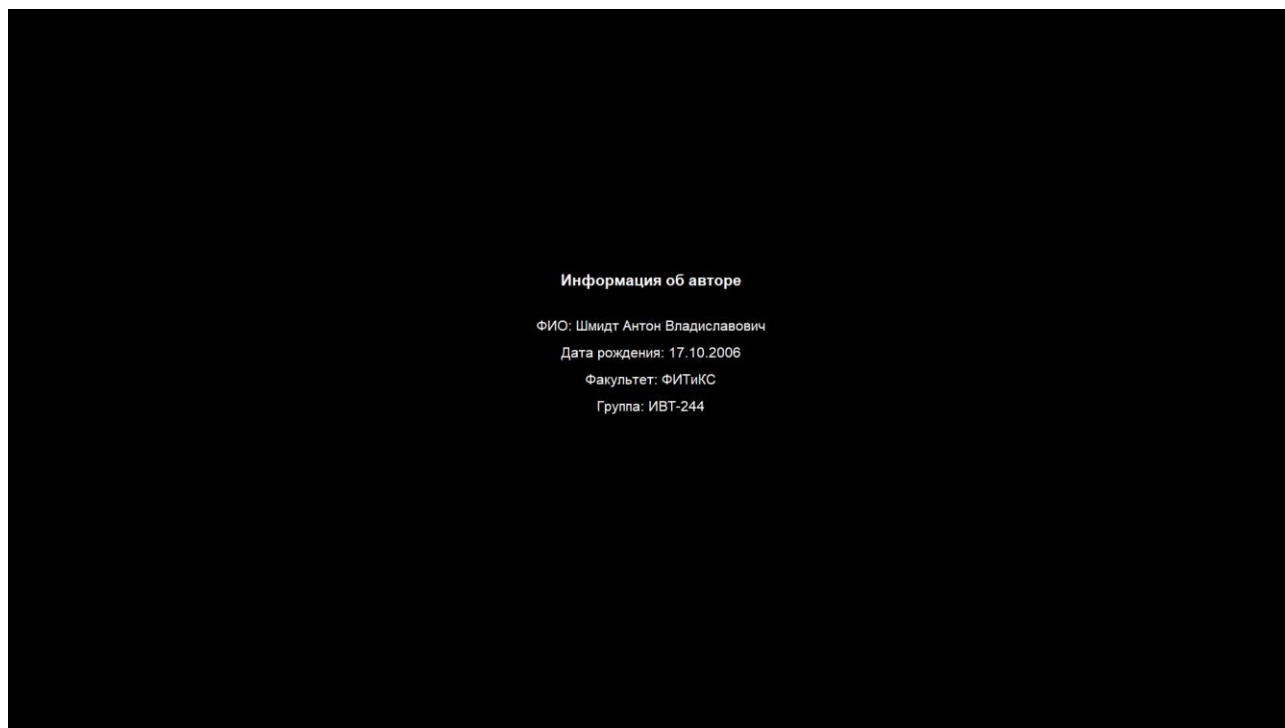


Рисунок 13 – вывод информации об авторе.

На рисунке 14 показан интерфейс звукового синтезатора, который позволяет воспроизводить звуки, соответствующие нотам, нажатием различных клавиш.

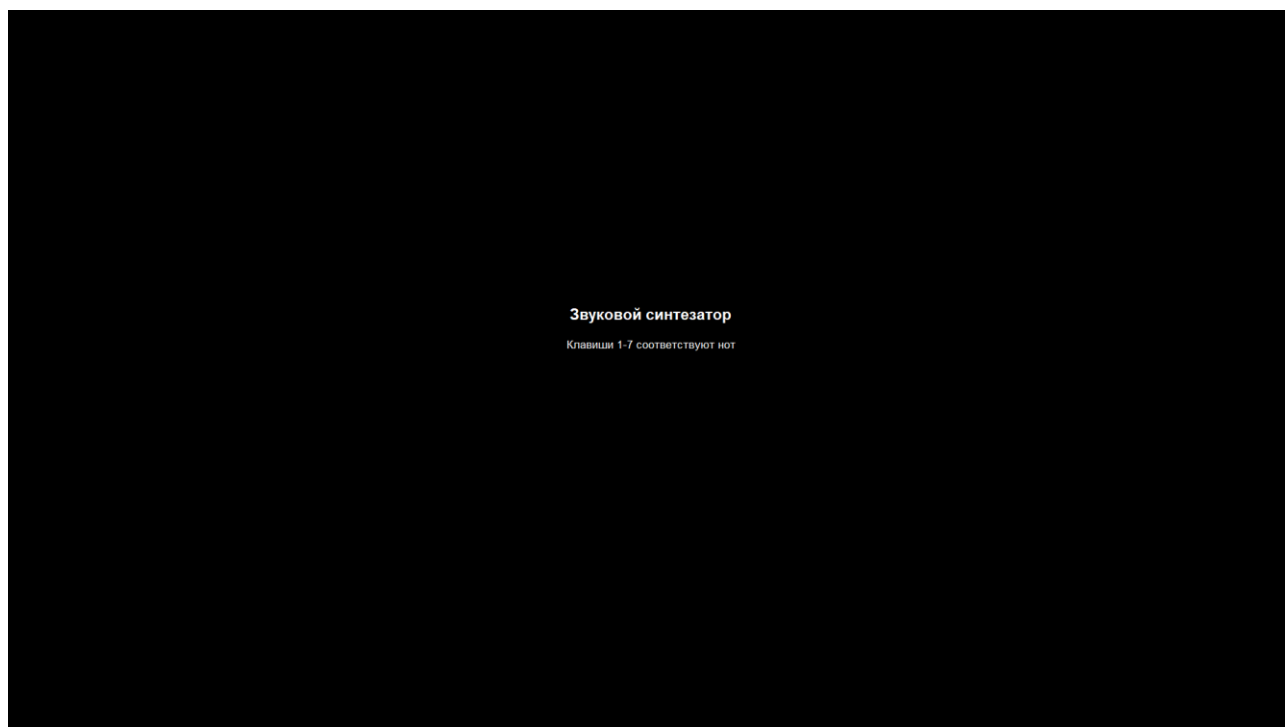


Рисунок 14 – интерфейс звукового синтезатора.

На рисунке 15 показан интерфейс игры «Морской бой» сразу после её запуска, когда пользователь ещё не предпринял никаких действий.

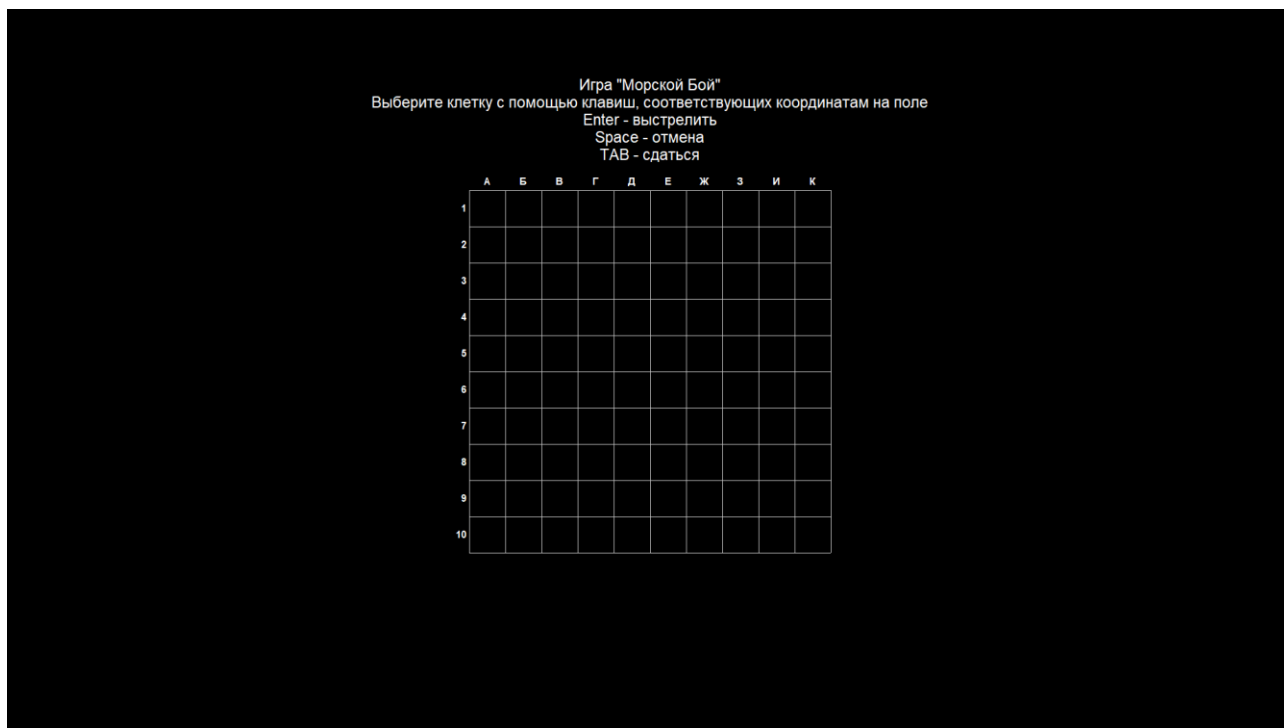


Рисунок 15 – начальное состояние игры «Морской бой».

На рисунке 16 показана игра «Морской бой» после того, как пользователь произвел 10 выстрелов по данным клеткам: А1, А2, А3, А6, А7, А8, А9, Г5, Г6 и Г7; но не совершил ни одного попадания по кораблю.

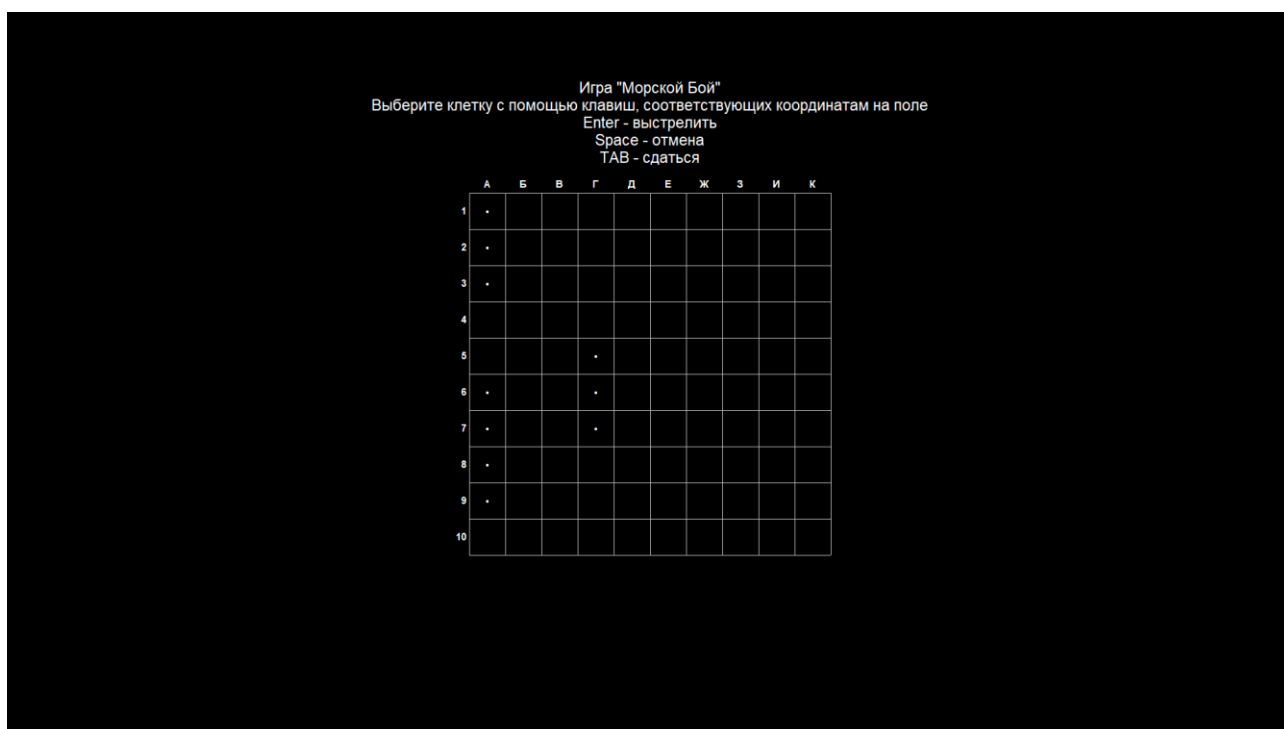


Рисунок 16 – интерфейс игры «Морской бой» во время игры.

На рисунке 17 приведён интерфейс игры после того, как пользователь нажал TAB и сдался, а на поле крестиками выделились положения всех кораблей.

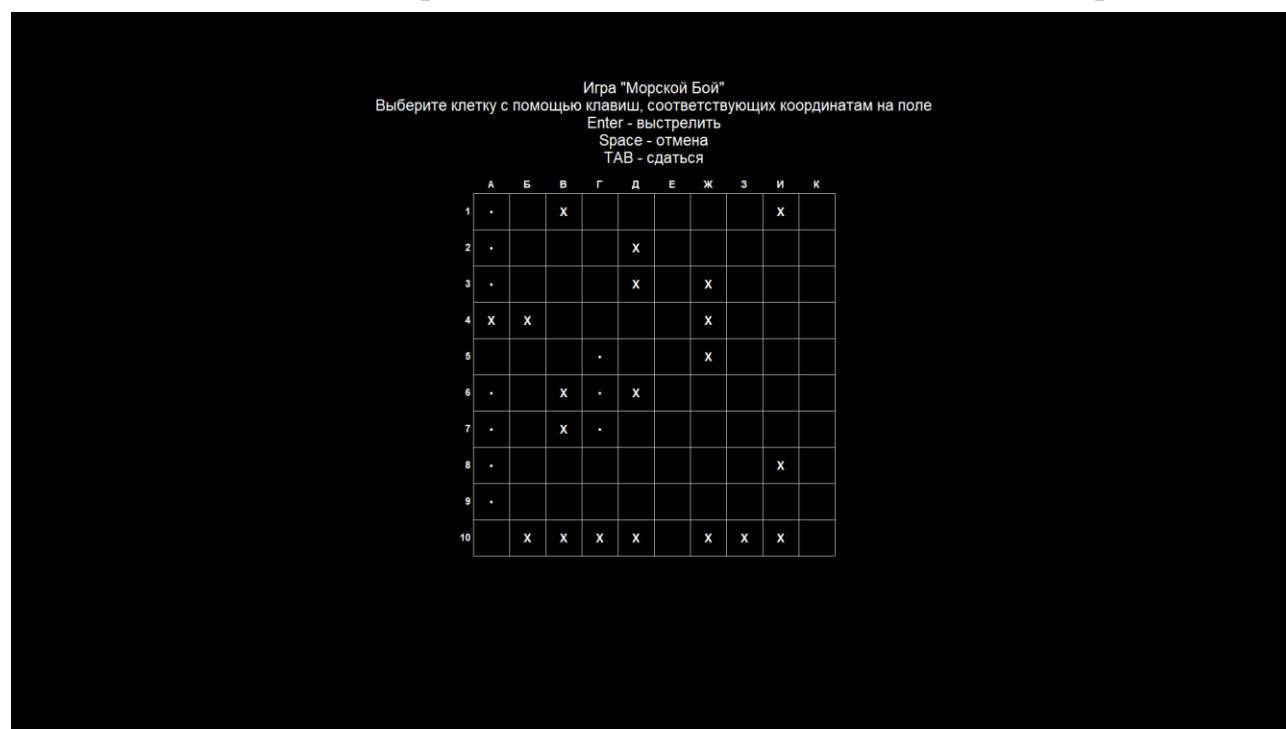


Рисунок 17 – вывод после раскрытие поля игры «Морской бой».

На рисунке 18 показан вывод после того, как пользователь выбрал пункт «5. Выход» в меню и нажал Enter.

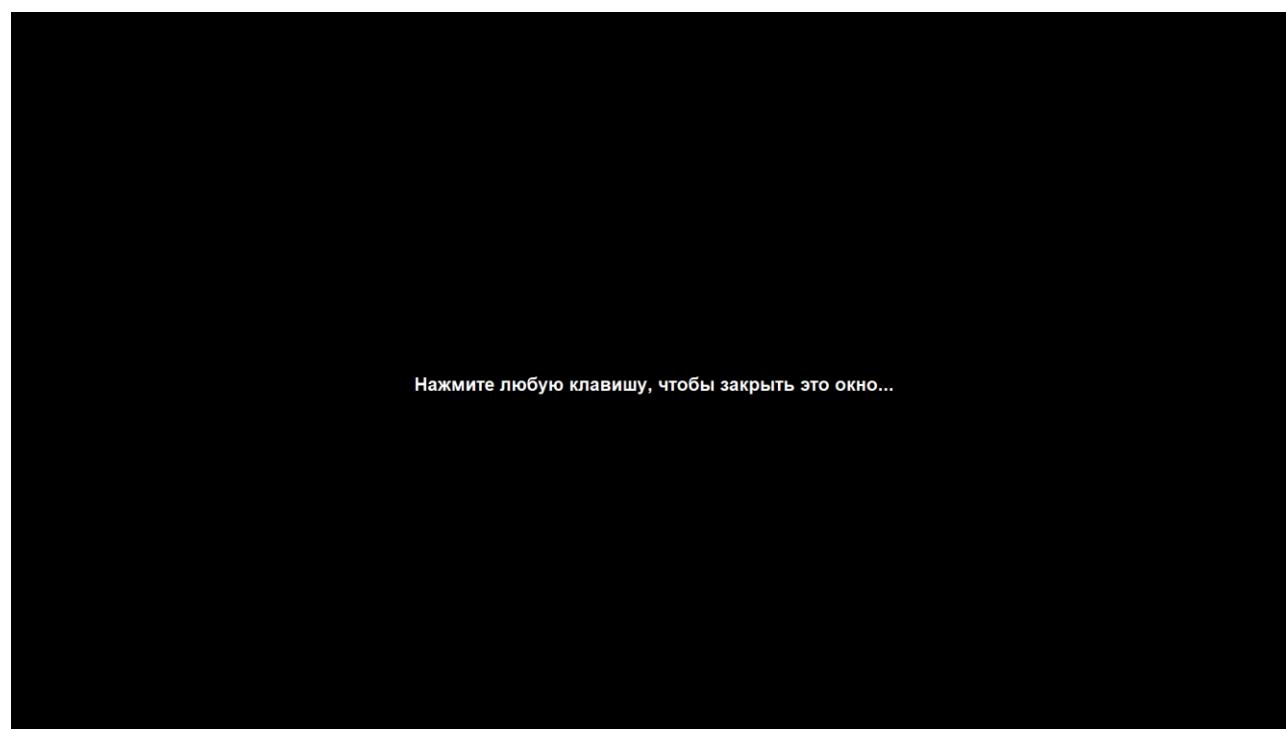


Рисунок 18 – вывод финального состояния при выходе.

Тестирование

Звуковой синтезатор:

Тестируемое действие	Ожидаемый результат	Полученный результат
Нажатие клавиш 1-7	Воспроизведение нот, соответствующих клавишам	Звуки необходимой частоты воспроизводятся
Нажатие клавиши ESC	Выход в главное меню	Программа возвращает пользователя в главное меню
Нажатие любых других клавиш	Ничего	Программа ничего не делает

Игра «Морской бой»:

Тестируемое действие	Ожидаемый результат	Полученный результат
Нажатие клавиш А-К	Выделение столбца на поле красным цветом И, если выделена строка, то выделение пересечения серым цветом	Столбец выделяется красным цветом При уже выделенной строке место пересечения выделяется серым цветом
Нажатие клавиш 1-0	Выделение строки на поле синим цветом И, если выделен столбец, то выделение пересечения серым цветом	Строка выделяется синим цветом При уже выделенном столбце место пересечения выделяется серым цветом
Нажатие клавиши Space	Сброс выделения клеток на поле, если были выделены строка, столбец или и строка, и столбец Ничего, если выделение отсутствовало	Выделение клеток пропадает с поля, если оно было Ничего не происходит, если клетки не были выделены

Нажатие клавиши TAB	Выделение на поле клеток, где находятся корабли символом «X» Выделение клеток не изменяется	На поле появляются символы «X» в тех местах, где расположены корабли Выделение клеток не изменяется
Нажатие клавиши Enter	Если выделена и строка, и столбец, то появление символа «X» или «·» в зависимости от наличия корабля на клетке, а выделение сбрасывается Если выделена только строка, только столбец или ничего не выделено, то ничего не происходит и выделение не сбрасывается	При выделенных и строке, и столбце появляется символ «X» или «·» в зависимости от попадания в корабль, а выделение пропадает с поля Если выделена только строка, только столбец или ничего не выделено, то на поле ничего не меняется
Генерация кораблей	Создание матрицы, которая хранит расположение 10 кораблей (4 по 1 клетке, 3 по 2, 2 по 3 и 1 на 4) Корабли не должны быть расположены рядом друг с другом в пределах одной клетки Каждая генерация должна быть случайной	Матрица с кораблями создаётся, все 10 кораблей расположены согласно правилу, что не соприкасаются друг с другом При перезапуске игры каждый раз расположение разное
Перезапуск игры	Корабли расположены в ином порядке, нежели при предыдущем запуске игры	Генерируются случайные наборы кораблей каждый раз

Нажатие ESC	Возврат в главное меню	Программа возвращает пользователя в главное меню
Нажатие любых других клавиш	Ничего	Программа ничего не делает

Список литературы

1. Березин, Б.И. Начальный курс Си и С++ / Б.И. Березин, С.И. Березин. – М.: Диалог-МИФИ, 1996. – 288 с.
2. ГОСТ 19.701-90. ЕСПД. Схемы алгоритмов, программ, данных и систем. – М.: Изд-во стандартов, 1991. – 26 с.
3. Крячков, А.В. и др. Программирование на С и С++. Практикум. / А.В. Крячков и др. – М.: Радио и связь, 1997. – 344 с.
4. Макогон, В.С. Язык программирования Си для начинающих / В.С. Макогон. – Одесса: НПФ "АСТРОПРИНТ", 1993. – 96 с.
5. Подбельский, В.В. Программирование на языке Си / В.В. Подбельский, С.С. Фомин. – М.: 2000 – 600 с.
6. Флоренсов, А.Н. Введение в программирование. Семантический подход: учеб. пособие / А.Н. Флоренсов. – Омск: Изд-во ОмГТУ, 1998. – 220 с.
7. Шафеева, О.П. Технологии программирования. С++: учеб. пособие / О.П. Шафеева. – Омск: Изд-во ОмГТУ, 2007. – 80 с.
8. Шилл, Г. Справочник программиста по С/С++. Учеб. Пособие / Г. Шилл – М.: Издательский дом «Вильямс», 2000. – 448 с.